

MBA 290T: Fundamental of Agentic AI

Haas School of Business,
UC–Berkeley Fall 2026

Co-Instructors

Alexandre Mas

Email: amas@berkeley.edu

Office Hours: By appointment only

Pepe Alonso

Email: pepe@berkeley.edu

Office Hours: By appointment only

Class Schedule

Lectures: **Tuesdays 4:00 – 6:00 pm PT in N570**

First class: **Tuesday, August 25, 2026**

Last class: **Tuesday, October 6, 2026**

Final assignment (Assignment 5) due: **Tuesday, October 13, 2026**

Seven Tuesday sessions, beginning with Class 1 on 8/25 and ending with Class 7 on 10/6.

There is **no final exam** and no final presentation. The course is assessed entirely through attendance and five graded assignments.

Course Objective

Fundamental of Agentic AI is not a survey course about AI, and it is not a prompt-engineering workshop. In seven sessions we go from “what is a variable” to building and evaluating a tool-using AI agent — covering programming foundations, full-stack software systems, machine learning, deep learning and transformers, LLM behavior and retrieval, and agent architecture.

The focus is **conceptual fluency that survives the tools**. Models and frameworks change every quarter; the mechanisms underneath them do not. You will leave able to build, evaluate, and lead technical AI work — not just talk about it.

Concretely, the course targets **Level 2 fluency**: you do not need a PhD to reason about AI systems, but knowing how to use ChatGPT is not a differentiator either. Level 2 is understanding how these systems actually work well enough to make good decisions about them.

Learning Outcomes

A student who successfully completes this course will be able to:

1. Read, modify, test, and debug real code, and explain what a program does before running it.
2. Describe how a production software system fits together — frontend, backend, API, database, authentication, secrets, testing, and deployment — and ship one end to end.
3. Decide when a problem calls for explicit rules versus learning from examples, and interrogate a dataset for leakage, missingness, and bad splits before trusting a model built on it.
4. Explain training, loss, gradient descent, overfitting, and regularization, and read an evaluation as a business conversation about thresholds, error costs, and decision rights.

5. Explain embeddings, attention, and the transformer block concretely enough to teach them, and connect them to how LLMs actually behave.
6. Design prompts as product interfaces, ground a system in retrieval, and treat “no answer found” as a successful result.
7. Build a bounded single agent with explicit tool schemas, a permission policy enforced in code, a stopping rule, evaluations, and a trace.
8. Judge when a multi-agent architecture earns its coordination cost — and when it does not.

Prerequisites

None. This course assumes no programming background. Class 1 starts at “what is a variable.”

You do need to bring a laptop you can install software on (macOS, Windows, or Linux), and a willingness to work in a terminal. Everything else is taught.

Course Website

All slides, live in-browser demos, diagrams, and starter code are published as a public website and remain available after the course ends. Slides are public by design — the goal is that you can always come back to a concept later.

Course site: <https://haas-ai-classes-fall-26.vercel.app>

Assignment submissions: [bCourses / submission portal URL]

Reading Materials

There is no required textbook. All material is self-contained in the slides, live demos, and in-class work. Optional deeper reading is linked from the slides where relevant (including Karpathy’s LLM Wiki gist, “Attention Is All You Need,” and Anthropic’s engineering write-ups on agents and evaluation).

Course Format

Each session is two hours, which is deliberately tight for the amount of material. Three things follow from that:

- **Live, hands-on sessions.** Most slides contain runnable code that executes in your browser. We build things during class rather than watching slides.
- **Self-study slides.** Classes 2, 6, and 7 carry more material than two hours comfortably fits. Slides marked with a **SELF-STUDY** badge are the ones I skip when time runs short. They stay in the published deck, and they are fair game for your assignments. Read them.
- **Assignments are where the learning consolidates.** The two ungraded in-class exercises are small; the five graded assignments are real builds.

Complete the Pre-Class Requirements below before 8/25 so Class 1 can start with code rather than installation.

Pre-Class Requirements

Have all of this working before Class 1 on 8/25. If any of it gives you trouble, contact an instructor before the first class.

Accounts — all free to create, and all three can sign in with GitHub:

- **GitHub** — you will push every assignment here
- **Vercel** — deployment for web projects
- **Supabase** — database and authentication for web projects

An AI coding agent — pick one and install it. We use these in every single class:

- **Claude Code**, or

- **OpenAI Codex CLI**, or
- **Google Antigravity**

On your laptop:

- **Python 3.10+**
- **Git**
- A code editor — **VS Code** or similar
- **Obsidian** — free, used in a later assignment

Costs. The free tiers of GitHub, Vercel, and Supabase are sufficient for everything we do. An AI coding agent subscription runs about **\$20/month**, and Assignments 4 and 5 use roughly **\$5–20** of LLM API credit in total. Budget **\$25–45** for the course. If cost is a barrier, contact me before Class 1 — this is solvable and I would rather solve it than have you fall behind.

One assignment trains a neural network on your laptop. It runs on Apple Silicon, NVIDIA GPUs, and plain CPUs — no cloud compute required — but a CPU-only machine will take several hours. Plan accordingly.

Attendance

Attendance is required and is worth 20% of your grade.

Much of this course happens in the room: live coding, debugging together, and working through exercises. Slides alone are a poor substitute.

You get **one absence with no penalty**, no explanation required. Beyond that, each missed session reduces the attendance component. If you know in advance you will miss more than one session, talk to me early.

Graded Assignments

Every class ends with an implementable build. **Five of them are graded assignments.** Assignments 1–4 are the end-of-class deliverables for Classes 2 through 5. Assignment 5 is the course capstone and spans Classes 6 and 7.

#	Assigned	Due (11:59 pm PT)
1	Class 2 — 9/1	Tue 9/8
2	Class 3 — 9/8	Tue 9/15
3	Class 4 — 9/15	Tue 9/22
4	Class 5 — 9/22	Tue 9/29
5	Classes 6 & 7 — 9/29 & 10/6	Tue 10/13

Assignments 1–4 are due **one week after they are assigned: the following Tuesday at 11:59 pm PT**. Assignment 5 spans two classes — part 1 is assigned in Class 6 and part 2 in Class 7 — and the combined deliverable is due **one week after Class 7: Tuesday 10/13 at 11:59 pm PT**. This creates a consistent weekly submission schedule from 9/8 through 10/13.

Class 1 has an in-class exercise that is not graded, as do the prediction-model exercise inside Class 3 and the attention-by-hand exercise inside Class 4. They still matter — but you are not submitting them.

The content of each assignment will be specified separately. This syllabus fixes the number of assignments, when each is assigned, when each is due, and how each is weighted. The scope, deliverables, and required evidence for every assignment will be published in a dedicated assignment brief and announced in class ahead of time.

Assignments are **individual**. You are expected to use AI coding agents (see the AI policy below); you are also expected to understand and be able to explain everything you submit.

What to Submit

Each submission should make it easy to understand, inspect, and try the work. In general, submit:

1. **A short delivery note** explaining what you made, the important choices you made, and any known limitations.
2. **A link to the GitHub repository** containing the relevant code and files.
3. **Evidence that the result works**, in the form appropriate to the assignment. This may be a public URL, a local demonstration, a recording, model outputs, plots, an agent trace, or other supporting files.

There is no required file name or universal submission template. The assignment brief for each assignment will state the current scope and any assignment-specific materials to include.

How Assignments Are Graded

Every assignment is scored out of 10 using the same broad framework:

Criterion	What I am looking for	Pts
Deliverable quality	The relevant code, files, structure, and technical choices are appropriate for the assignment and are explained clearly	4
Testing & evaluation	The submission includes appropriate evidence used to check, evaluate, or challenge the work	3
Working result	The result can be inspected or demonstrated and meaningfully does what the assignment is intended to accomplish	3

This is a shared grading framework, **not a fixed feature checklist**. What counts as strong code, appropriate testing, or a working result will depend on the assignment — a deployed URL, evaluation results, training evidence, model outputs, or a live demonstration may each be the right evidence for a given assignment.

The instructor may adjust the exact deliverable, required evidence, or emphasis within these categories as the course, tools, and projects evolve. The assignment brief provided for each assignment is the source of truth, and any changes will be communicated clearly in advance.

Partial credit is normal and expected. **Something half-working with an honest write-up explaining what broke will score better than something polished you cannot explain.** That is the whole point of the course.

Grading

Performance will be graded as follows:

Component	Weight
Attendance	20%
Assignments 1–4 (lowest score dropped)	50%
Assignment 5	30%
Total	100%

Each assignment is scored out of 10 points using the shared framework above, applied to the specific assignment brief. For Assignments 1–4, the lowest of the four scores is dropped and the remaining three scores are combined equally to make 50% of the final grade. Assignment 5 makes up 30%; attendance makes up 20%. You get a pass on one missed session in the attendance grade, so you are not penalized for missing a single class.

Each assignment will have a current Definition of Done in its assignment brief or class deck. That brief may be updated as the course develops; clearly announced updates supersede this syllabus overview. The shared grading categories and the assignment’s overall weight remain the consistent structure.

Grade Dispute Policy

If you would like an assignment re-graded, submit a written petition within 72 hours of receiving the grade. The petition should identify the specific issue, be well reasoned, and be word-processed. Submit it together with the assignment. I reserve the right to re-grade the whole assignment.

Honor Code

As members of the UC Berkeley community, you are expected to adhere rigorously to the UC Berkeley Honor Code. Submitting work you cannot explain, or work produced by another student, is a violation and will be reported to the Center for Student Conduct.

If I have reason to doubt that submitted work is your own, I may ask you to explain it — verbally, live, on any part of the submission. This is the natural check for a course where AI assistance is expected: the tools can write the code, but they cannot sit in the meeting for you.

For further information: <https://asuc.org/honorcode/>

AI / LLM Policy

AI use is not merely permitted in this course — it is required.

Every assignment is designed to be built with an AI coding agent. Refusing to use AI will make this course harder, not more honest.

What is required alongside it:

- **You must understand and be able to explain everything you submit.** Every line, every file, every schema decision. If you cannot explain it, you did not do the assignment — you watched someone else do it.
- **Review the diff before you accept it.** This is an expected part of working responsibly with an AI coding agent, and you may be asked to explain your review.
- **Cite nothing, disclose everything.** You do not need to footnote AI usage. You do need to be honest if asked what you wrote versus what an agent wrote.
- **Never paste confidential or personal data into a model.** This applies to any assignment where you choose your own source material.

The distinction this course cares about is not “did a human or an AI write this.” It is “**does the person submitting it understand the system well enough to be responsible for it.**” That is the entire professional skill being taught. The shared grading framework rewards explainable technical choices, appropriate validation, and a result that works.

Accessibility and Accommodations

If you have a documented disability and need accommodations, please contact the Disabled Students’ Program (DSP) and let me know as early as possible so we can make arrangements. Requests are handled confidentially.

If anything about the course format — the pace, the terminal work, the live coding — is a barrier for you, tell me. It is usually fixable.

Course Communication

Email is the fastest way to reach me: pepe@berkeley.edu. I aim to respond within 24 hours on weekdays.

For “my code is broken” questions, include: what you expected, what happened, the full error message, and what you already tried. That is not bureaucracy — it is the debugging discipline from Class 1, and writing it out solves the problem about a third of the time.

What Each Class Covers

Class 1 — Code & Programming Foundations. What code actually is and how it runs. Variables, types, conditionals, loops, and functions. Lists vs. dictionaries, and why data shape is a management decision. Errors, tracebacks, and debugging as hypothesis testing. Tests as executable promises. Programming languages and why Python. The terminal. Clean code: readability as throughput, the WTF-per-minute metric, guard clauses, magic numbers, and comments that explain why. Organizing a project across files, with an entry point that orchestrates and separation of concerns. Git, GitHub, branches, diffs, and pull requests. AI coding agents: plan mode, YOLO mode, and when not to reach for the agent.

Class 2 — Software Systems. Frontend: HTML, CSS, JavaScript, the DOM, components, and state. Backend: servers, APIs, HTTP methods, status codes, and JSON. Databases: tables, relationships, SQL, transactions, migrations, indexes, and caching. Environment variables and secrets, and why a frontend cannot keep one. Authentication vs. authorization, and OAuth. Cloud platforms and latency. Testing and the test pyramid. CI, deployment, environments, rollback, logs, and alerts. Tech stacks and the tradeoffs behind them.

Class 3 — Machine Learning Foundations. When to write rules and when to learn them from examples. Classification, regression, and ranking. Data as the product before the model: labels, leakage, splits, and missingness. Loss and gradient descent. Generalization, overfitting, underfitting, bias-variance, regularization, and cross-validation. The classical toolbox: linear models, decision trees, random forests, gradient boosting, kNN, and SVMs. Feature engineering. Unsupervised learning: clustering, dimensionality reduction, anomaly detection. Reinforcement learning: states, actions, rewards, the Bellman equation, and reward hacking. Evaluation as a business conversation: confusion matrices, precision, recall, thresholds, and calibration. Fairness, causality, and decision rights.

Class 4 — Deep Learning & Transformers. Neurons, activations, and why depth matters. Forward pass, loss, backpropagation, optimizers, batches, and epochs. CNNs and the GPU story. Teaching machines to read: embeddings, tokenization, and BPE. Sequences and attention — query, key, value, softmax, and multi-head. The transformer block, positional encoding, and residual connections. Scaling laws, pretraining, transfer learning, fine-tuning, and alignment. How we got from transformers to LLMs — and how to build and train a small one yourself.

Class 5 — LLM Behavior, Prompting & Retrieval. LLMs as probability distributions, and temperature as the steering dial. Fine-tuning, LoRA, and prompting — three levers, three jobs. Prompts as product interfaces: specifying the job, separating instructions from data, instruction hierarchy as a trust boundary, and output format as a reliability tool. Context windows, context rot, and compression. System prompts and `CLAUDE.md`. Thinking models. Hallucination as a system property, and four defenses. Retrieval-augmented generation: chunking, metadata, grounding, and “no answer found” as success. Embeddings, vector databases, and hybrid search.

Class 6 — Production LLMs & Single Agents. Where an LLM lives: hosted, self-hosted, or local. Budgets, timeouts, and retries. Structured output as an interface, and validating before acting. What an agent actually is: the loop, state, tools, and a stopping rule. Tool allowlists, permission policies, and why the model must never enforce its own guardrails. Stochastic agents vs. deterministic workflows — the skeleton-and-spark pattern. Designing tools like public APIs. MCP and skills. Orchestration patterns: prompt chaining, routing, and evaluator-optimizer. Evaluations, traces, and diagnosability. Cost engineering: token budgeting, prompt caching, model routing, and batch APIs.

Class 7 — Multi-Agent Systems, Ops & Safety. Where single agents hit walls. The three axes of “more

than one”: specialization, coordination, and communication. Swarm intuitions and the credit assignment problem. Canonical patterns: orchestrator-workers, fan-out, subagents, and the advisor strategy. Managed agents, pets vs. cattle. Knowledge architecture as coordination architecture. Production ops: cron and webhook triggers, observability, run records, and the honest cost curve. Safety: alignment failures, context rot, prompt injection and the confused deputy, and the OWASP LLM Top 10. Accountability, procurement, and regulation. When *not* to go multi-agent.

Class Schedule

Class	Date	Topic	Deliverable
1	Tue 8/25	Code & Programming Foundations	In-class exercise (<i>not graded</i>)
2	Tue 9/1	Software Systems	Assignment 1 — due Tue 9/8
3	Tue 9/8	Machine Learning Foundations	Assignment 2 — due Tue 9/15 (prediction model exercise, in class, not graded)
4	Tue 9/15	Deep Learning & Transformers	Assignment 3 — due Tue 9/22 (attention by hand, in class, not graded)
5	Tue 9/22	LLM Behavior, Prompting & Retrieval	Assignment 4 — due Tue 9/29
6	Tue 9/29	Production LLMs & Single Agents	Assignment 5, part 1
7	Tue 10/6	Multi-Agent Systems, Ops & Safety	Assignment 5, part 2 — all due Tue 10/13

The instructor reserves the right to amend this syllabus. Students will be notified of any changes.